

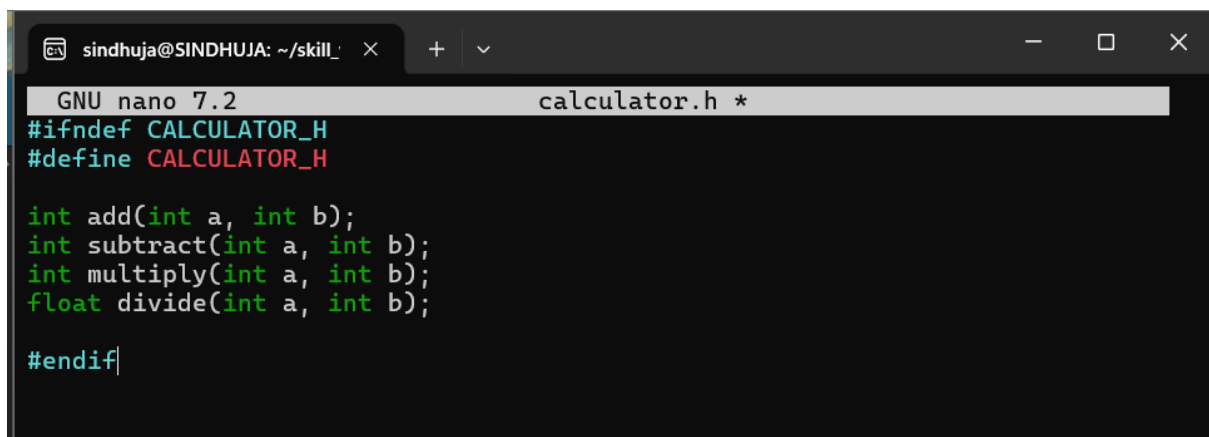
SKILL WEEK 11

S21: Question1

Aim:

To connect multiple program modules, validate interfaces, test end-to-end execution, handle errors, improve reliability, and refactor components using C programs in Linux.

1. Nano calculator.h



```
GNU nano 7.2 calculator.h *
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);
float divide(int a, int b);

#endif
```

2. Nano calculator.c



```
GNU nano 7.2 calculator.c *
#include <stdio.h>
#include "calculator.h"

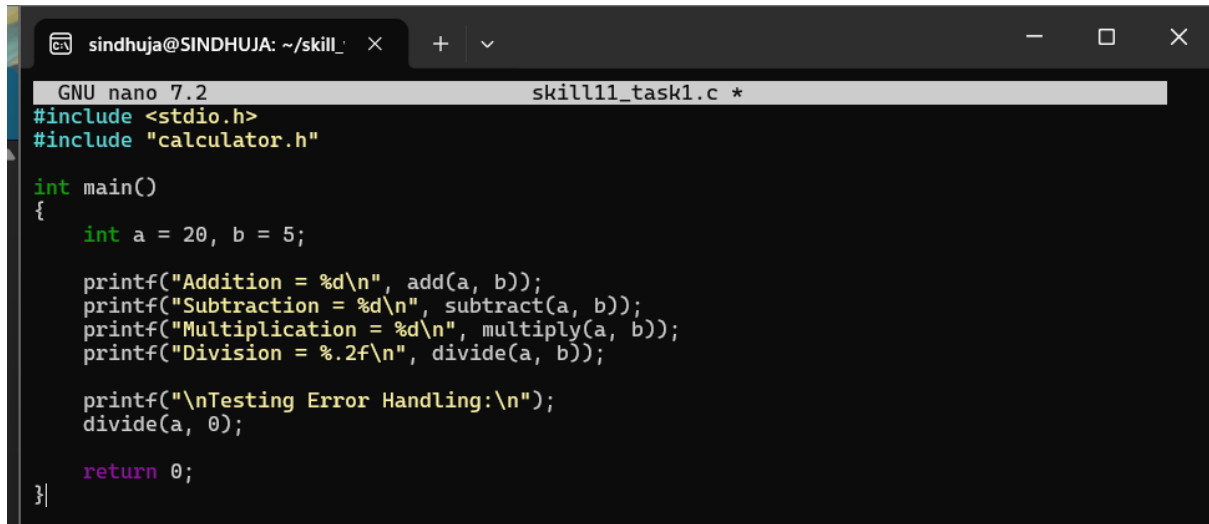
int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}

int multiply(int a, int b)
{
    return a * b;
}

float divide(int a, int b)
{
    if (b == 0)
    {
        printf("Error: Division by zero is not allowed.\n");
        return 0;
    }
    return (float)a / b;
}
```

Program:



```
sindhuj@SINDHUJA: ~/skill_ x + v
GNU nano 7.2 skill11_task1.c *
#include <stdio.h>
#include "calculator.h"

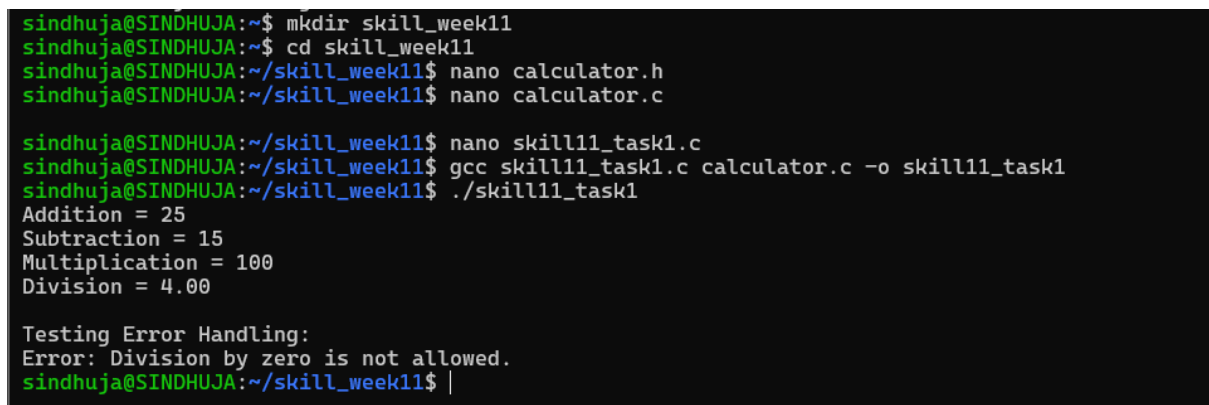
int main()
{
    int a = 20, b = 5;

    printf("Addition = %d\n", add(a, b));
    printf("Subtraction = %d\n", subtract(a, b));
    printf("Multiplication = %d\n", multiply(a, b));
    printf("Division = %.2f\n", divide(a, b));

    printf("\nTesting Error Handling:\n");
    divide(a, 0);

    return 0;
}
```

Output:



```
sindhuj@SINDHUJA:~$ mkdir skill_week11
sindhuj@SINDHUJA:~$ cd skill_week11
sindhuj@SINDHUJA:~/skill_week11$ nano calculator.h
sindhuj@SINDHUJA:~/skill_week11$ nano calculator.c

sindhuj@SINDHUJA:~/skill_week11$ nano skill11_task1.c
sindhuj@SINDHUJA:~/skill_week11$ gcc skill11_task1.c calculator.c -o skill11_task1
sindhuj@SINDHUJA:~/skill_week11$ ./skill11_task1
Addition = 25
Subtraction = 15
Multiplication = 100
Division = 4.00

Testing Error Handling:
Error: Division by zero is not allowed.
sindhuj@SINDHUJA:~/skill_week11$ |
```

Observation:

1. Multiple modules were connected using a header file.
2. Interfaces were validated through function declarations.
3. All modules were compiled together successfully.
4. The program executed all arithmetic operations correctly.
5. Division-by-zero was handled without crashing the program.

Result:

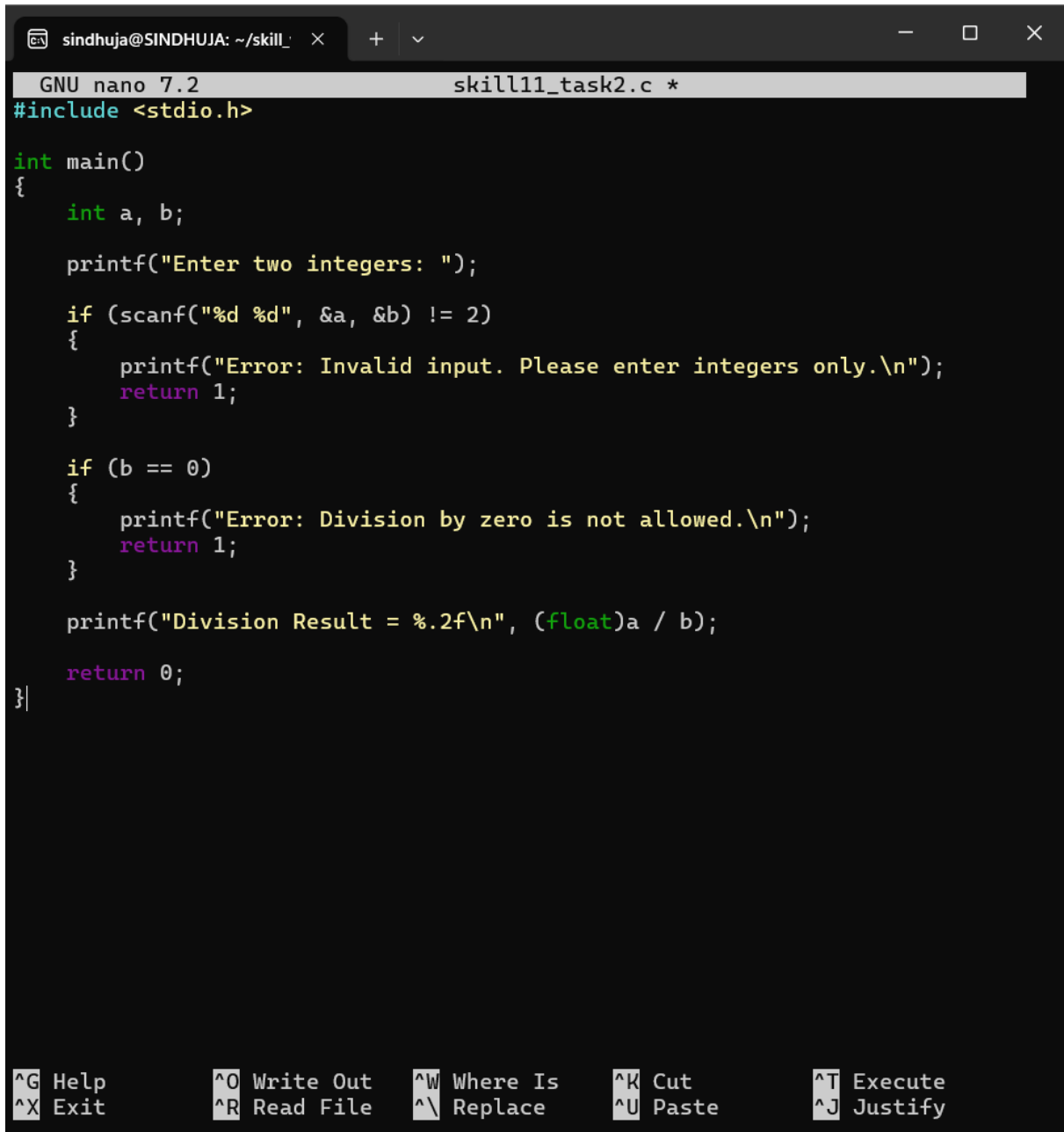
Successfully connected multiple modules, validated interfaces, tested end-to-end execution, handled errors, improved reliability, and demonstrated modular programming in Linux.

S21: Question2

Aim:

To detect runtime errors, generate error messages, log failures, handle invalid input, recover gracefully, and test different error scenarios using C programs in Linux.

Program:

A screenshot of a terminal window with a dark background. The window title bar shows 'sindhuj@SINDHUJA: ~/skill_'. The terminal header indicates 'GNU nano 7.2' and the file name 'skill11_task2.c *'. The code is a C program for division with error handling. It includes <stdio.h> and has a main function. It prompts the user to enter two integers. If scanf fails (returns != 2), it prints an error message and returns 1. If the second integer is zero, it prints a division by zero error and returns 1. Otherwise, it calculates the division result as a float and prints it. The bottom of the screen shows nano editor shortcuts: ^G Help, ^X Exit, ^O Write Out, ^R Read File, ^W Where Is, ^\ Replace, ^K Cut, ^U Paste, ^T Execute, ^J Justify.

```
sindhuj@SINDHUJA: ~/skill_
GNU nano 7.2 skill11_task2.c *
#include <stdio.h>

int main()
{
    int a, b;

    printf("Enter two integers: ");

    if (scanf("%d %d", &a, &b) != 2)
    {
        printf("Error: Invalid input. Please enter integers only.\n");
        return 1;
    }

    if (b == 0)
    {
        printf("Error: Division by zero is not allowed.\n");
        return 1;
    }

    printf("Division Result = %.2f\n", (float)a / b);

    return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

Output:

```
from ~/Sindhujas/valgrind-1.20.1
sindhuja@SINDHUJA:~$ cd ~/skill_week11
sindhuja@SINDHUJA:~/skill_week11$ nano skill11_task2.c
sindhuja@SINDHUJA:~/skill_week11$ gcc skill11_task2.c -o skill11_task2
sindhuja@SINDHUJA:~/skill_week11$ ./skill11_task2
Enter two integers: 20 5
Division Result = 4.00
sindhuja@SINDHUJA:~/skill_week11$ |
```

Observation:

1. The program successfully detected valid and invalid user inputs.
2. Runtime errors such as division by zero were handled safely.
3. Error messages were displayed for invalid input and invalid operations.
4. The program recovered gracefully without crashing during error scenarios.

Result:

Successfully detected runtime errors, generated error messages, handled invalid input, recovered gracefully, and tested different error scenarios using C programming in Linux.

S22: Question1

Aim:

To use Valgrind to detect memory leaks, identify memory errors, release allocated memory, verify cleanup, and document the findings in Linux.

Program:

```
sindhuja@SINDHUJA: ~/skill_ x + v
GNU nano 7.2 skill12_task1.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *arr = (int *)malloc(5 * sizeof(int));

    if (arr == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        arr[i] = i + 1;
        printf("%d ", arr[i]);
    }

    printf("\nMemory allocated successfully.\n");

    /* Release allocated memory */
    free(arr);

    printf("Memory released successfully.\n");

    return 0;
}
```

[^]G Help [^]O Write Out [^]W Where Is [^]K Cut [^]T Execute
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste [^]J Justify

Output:

```
sindhuja@SINDHUJA: ~/skill_ x + v
sindhuja@SINDHUJA:~$ mkdir skill_week12
sindhuja@SINDHUJA:~$ cd skill_week12
sindhuja@SINDHUJA:~/skill_week12$ nano skill12_task1.c
sindhuja@SINDHUJA:~/skill_week12$ gcc skill12_task1.c -o skill12_task1
sindhuja@SINDHUJA:~/skill_week12$ ./skill12_task1
1 2 3 4 5
Memory allocated successfully.
Memory released successfully.
```

```
sindhuja@SINDHUJA: ~/skill_ + v  
sindhuja@SINDHUJA:~/skill_week12$ sudo apt update  
[sudo] password for sindhuja:  
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease  
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]  
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]  
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1053 kB]  
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]  
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1319 kB]  
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [219 kB]  
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46.5 kB]  
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [299 kB]  
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1220 kB]  
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]  
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [245 kB]  
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1697 kB]  
Get:14 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.3 kB]  
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [1535 kB]  
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [340 kB]  
Get:17 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]  
Get:18 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [1654 kB]  
Get:19 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [353 kB]  
Get:20 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [11.3 kB]  
Get:21 http://archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [377 kB]  
Get:22 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [45.7 kB]  
Get:23 http://archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [13.1 kB]  
Get:24 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]  
Get:25 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [40.6 kB]  
Get:26 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5760 B]  
Get:27 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [31.0 kB]  
Get:28 http://archive.ubuntu.com/ubuntu noble-backports/universe Translation-en [18.6 kB]  
Get:29 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [12.6 kB]  
Fetched 11.6 MB in 23s (511 kB/s)  
Reading package lists... Done  
Building dependency tree... Done http://archive.ubuntu.com/ubuntu  
Ctrl+Click to follow link  
Reading state information... Done  
248 packages can be upgraded. Run 'apt list --upgradable' to see them.  
sindhuja@SINDHUJA:~/skill_week12$ sudo apt install valgrind  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
valgrind is already the newest version (1:3.22.0-0ubuntu3).  
0 upgraded, 0 newly installed, 0 to remove and 248 not upgraded.  
sindhuja@SINDHUJA:~/skill_week12$ valgrind ./skill12_task1  
==910== Memcheck, a memory error detector  
==910== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.  
==910== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info  
==910== Command: ./skill12_task1  
==910==  
1 2 3 4 5
```

```
1 2 3 4 5  
Memory allocated successfully.  
Memory released successfully.  
==910==  
==910== HEAP SUMMARY:  
==910==    in use at exit: 0 bytes in 0 blocks  
==910==   total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated  
==910==  
==910== All heap blocks were freed -- no leaks are possible  
==910==  
==910== For lists of detected and suppressed errors, rerun with: -s  
==910== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)  
sindhuja@SINDHUJA:~/skill_week12$
```

Observation:

1. Memory was allocated dynamically using malloc().
2. The allocated memory was released using free().
3. Valgrind analyzed heap memory usage.
4. No memory leaks were detected.
5. The program completed with 0 memory errors.

Result:

Successfully used Valgrind to detect memory leaks, verify memory cleanup, release allocated memory, and confirm that the program executed without memory errors.

S22: Question2

Aim:

To design test cases, create a test script, automate program execution, verify outputs, compare expected and actual results, and generate a test report in Linux.

Program:

A screenshot of a terminal window with a dark background. The window title bar shows 'sindhuja@SINDHUJA: ~/skill_'. The terminal content shows the GNU nano 7.2 editor editing a file named 'skill12_task2.c'. The code is a C program for adding two numbers. It includes <stdio.h>, defines an 'add' function that returns the sum of two integers, and a 'main' function that prompts the user to enter two numbers, reads them, calculates the sum using the 'add' function, and prints the result. The code is as follows:

```
GNU nano 7.2 skill12_task2.c *
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}

int main()
{
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("Sum = %d\n", add(a, b));

    return 0;
}
```

```
sindhuja@SINDHUJA: ~/skill_ x + v
GNU nano 7.2 test_cases.sh *
#!/bin/bash

echo "Test Case 1: 10 + 20"
echo "10 20" | ./skill12_task2

echo "Test Case 2: 50 + 30"
echo "50 30" | ./skill12_task2

echo "Test Case 3: -5 + 15"
echo "-5 15" | ./skill12_task2
```

Output:

```
sindhuja@SINDHUJA: ~/skill_ x + v
sindhuja@SINDHUJA:~$ cd skill_week12
sindhuja@SINDHUJA:~/skill_week12$ nano skill12_task2.c
sindhuja@SINDHUJA:~/skill_week12$ gcc skill12_task2.c -o skill12_task2
sindhuja@SINDHUJA:~/skill_week12$ nano test_cases.sh
sindhuja@SINDHUJA:~/skill_week12$ chmod +x test_cases.sh
sindhuja@SINDHUJA:~/skill_week12$ ./test_cases.sh
Test Case 1: 10 + 20
Enter two numbers: Sum = 30
Test Case 2: 50 + 30
Enter two numbers: Sum = 80
Test Case 3: -5 + 15
Enter two numbers: Sum = 10
sindhuja@SINDHUJA:~/skill_week12$ ./test_cases.sh > report.txt
sindhuja@SINDHUJA:~/skill_week12$ cat report.txt
Test Case 1: 10 + 20
Enter two numbers: Sum = 30
Test Case 2: 50 + 30
Enter two numbers: Sum = 80
Test Case 3: -5 + 15
Enter two numbers: Sum = 10
sindhuja@SINDHUJA:~/skill_week12$ |
```

Observation:

1. Multiple test cases were designed for the addition program.
2. A shell script automated the execution of all test cases.
3. The outputs matched the expected results for each test case.
4. The execution results were stored in a report file.
5. The report was verified using the cat command.

Result:

Successfully designed test cases, created a shell script, automated program execution, verified outputs, compared results, and generated a test report in Linux.